

FastAPI

웹서비스 개발의 역사와 동향

- 1세대 : 정적 웹 페이지
 - HTML/CSS로 만들어진 페이지로 구성
 - 서버는 사용자의 요청에 따라 정적인 페이지 제공 역할만 했음
 - 모든 콘텐츠를 미리 제작해야 하고, 사용자 별로 다른 정보를 표시하기 어렵다는 단점
- 2세대 : 동적 웹 페이지
 - 사용자별로 다른 정보를 표시하거나, 실시간으로 변하는 정보를 제공
 - 개인화된 콘텐츠와 실시간 정보 제공하면서 웹 서비스의 활용 범위가 넓어짐
- 3세대 : MVC 프레임워크
 - 어플리케이션의 규모가 커지면서 코드의 구조화와 재사용성이 중요해지면서 MVC 패턴 기반의 웹 프레임워크가 등장
 - 어플리케이션을 모델 / 뷰 / 컨트롤러로 나누어 개발하는 방식으로 유연하고 재사용 가능한 구조로 개발
- 4세대 : API 중심 서비스
 - 다양한 백엔드 플랫폼 지원을 위한 마이크로 서비스 아키텍처(MSA)가 주를 이루고 있음
 - MSA : 서비스 세부 기능별로 컴퓨터를 분산하고, 서로 간에 API를 통해 통신하여 전체 서비스를 운영하는 아키텍처

프레임워크

● 프레임워크의 정의

- 특정 분야의 어플리케이션 개발을 돕기 위해 미리 만들어진 코드와 구조의 집합
- 개발자가 반복적으로 사용하는 기능을 제공하여 개발 생산성을 높임
- 대표적 웹 어플리케이션 프레임워크
 - 파이썬 : Django, Flask, FastAPI
 - 자바 : Spring, Spring Boot
 - 자바스크립트 : Node.js

● 프레임워크의 유형

- 풀스택 프레임워크
 - 장고나 스프링은 프레임워크 내에서 웹 어플리케이션의 모든 기능 지원
- 마이크로 프레임워크
 - Flask나 FastAPI는 백엔드 구현에 필요한 핵심 기능만 제공

마이크로 프레임워크

- 간결한 기능을 제공하므로 학습 곡선이 낮고 빠르게 사용 가능
- 필요한 기능만 골라서 사용할 수 있으므로 서비스가 경량화될 수 있음
- 프로그래밍 언어 기반 모든 기능을 그대로 프레임워크와 함께 사용 가능하므로 확장성이 높음
- 프레임워크 내에서 제공하는 핵심 기능 외에는 직접 작성한 코드이므로 이상 동작시에도 해결이 쉬움
- FastAPI는 파이썬 라이브러리와 결합하여 파이썬 생태계의 강력한 기능 활용 가능

FastAPI

● 개요

- 파이썬으로 구현된 고성능 웹 프레임워크로 주로 API 개발에 사용
- 비동기 처리를 강조하며, Starlette(비동기 프레임워크)와 Pydantic(데이터 검증 및 설정 라이브러리)에 기반을 둠

● 주요 특징

- 요청(request) 및 응답(response) 스키마 정의에 따라 Swagger UI 문서를 자동으로 생성
- 위 스키마를 명시적으로 정의함으로써 타입 안전(type safety) 개발이 가능
- ASGI(Asynchronous Server Gateway Interface)를 지원하므로 비동기 처리가 가능해 속도가 빠름

* ASGI

- 파이썬에서 비동기 웹 애플리케이션을 위한 표준 인터페이스
- 비동기적인 웹 서버와 애플리케이션을 연결하여 빠르고 실시간으로 작동하는 서비스를 구현할 수 있도록 도와 줌
- 이벤트 기반 처리를 지원하여 WebSocket과 같은 기술을 사용할 수 있게 해 주며, FastAPI와 같은 프레임워크에서 비동기 처리를 가능하게 함

FastAPI 시작하기

- FastAPI 설치하기
 - Python 3.6 이상 필요

```
(.venv)  
$ pip install fastapi
```

Hello, World! API 만들기

- main.py

```
1 from fastapi import FastAPI
2
3 app = FastAPI()
4
5 @app.get("/")
6 def read_root():
7     return {"message": "Hello, World!"}
```

서버 실행

- Uvicorn 설치

- `pip install uvicorn`

- FastAPI 어플리케이션을 로컬에서 실행할 수 있는 ASGI 서버 구현체
 - 비동기 웹 서버로, FastAPI의 비동기 처리 기능과 호환되어 효율적인 성능 발휘

- Uvicorn 실행

- `uvicorn main:app --reload --host 0.0.0.0 --port 8000`

- main : FastAPI 어플리케이션 코드가 있는 파이썬 파일명(main.py)
 - app : FastAPI 인스턴스 객체명(`app = FastAPI()`)
 - -- reload : 개발 중 코드 변경 시 자동으로 서버 재시작 • --host 0.0.0.0 : 외부에서 서버에 접근할 수 있도록 설정

서버 실행

- 서버 접속

- 명령어 실행 후, <http://127.0.0.1:8000> 주소로 브라우저에서 실행하여 FastAPI 어플리케이션이 접속

- FastAPI와 Uvicorn의 조합

- FastAPI는 Uvicorn을 포함한 다양한 ASGI 프레임워크와 호환
- Uvicorn의 비동기 처리 기능을 최대한 활용할 때 FastAPI는 최적의 성능을 발휘함
- 따라서 FastAPI와 Uvicorn의 조합이 가장 권장됨

자동 문서화

- 자동 문서화의 중요성

- 문서화의 부재 - 개발 과정에서 API 문서화는 시간이 많이 소요되고 지속적인 업데이트가 필요한 문제
- FastAPI는 자동 문서화 기능을 통해 해결함

- 장점

- 시간 절약
- 항상 최신 정보 유지
- API 테스트
- 타입 검사와 유효성 검증

자동 문서화

- 방식

- 타입 힌트와 데코레이터 활용 : 코드 내의 타입 힌트(Type Hint)와 데코레이터(decorator)를 사용하여 API 문서를 자동 생성
- 라우터를 통한 문서 생성 : 예를 들어 `@app.get("/")` 라우터를 통해 해당 API 엔드포인트에 대한 문서 자동 생성

- 사용법

- Swagger UI 접근
 - <http://127.0.0.1:8000/docs> 주소를 통해 Swagger UI 접근 가능
 - API 엔드포인트의 상세 정보 확인 및 테스트 수행 가능
- ReDoc 접근
 - <http://127.0.0.1:8000/redoc> 주소를 통해 ReDoc을 통한 다른 형태의 문서 확인 가능

Swagger UI(Docs) 와 ReDoc의 차이점

- Swagger UI(Docs)

- 대화형 API 테스트 가능
- 매개변수 형태 및 예시 표시
- OAuth2 인증 테스트 지원
- UI 커스터마이징 가능

- ReDoc

- 깔끔하고 직관적인 UI 제공
- 마크다운 문법 지원
- 단일 페이지 구조로 문서 전체 구조 쉽게 파악
- 대화형 테스트 불가능

자동 문서화의 주요 용어

- **curl** : 터미널(명령 프롬프트)에서 사용할 수 있는 명령 줄 도구로, 웹사이트나 API 서버와 통신할 수 있게 해줌
- **GET / Read Root** : 서버로부터 정보를 조회하는데 사용되는 문서화
- **200** : HTTP 응답 코드는 서버가 클라이언트의 요청에 어떻게 응답 했는지를 나타내는 숫자 코드로 요청이 성공적으로 완료되었음을 의미
- **application / json** : 데이터가 JSON 형식으로 변환되었음을 나타내는 MIME 타입
- **Response body** : 서버가 반환하는 실제 데이터 부분
- **Response headers** : 서버 응답에 대한 메타데이터(정보의 정보)를 포함